CharLES Manual for Technion Flow Physics Lab

Mordechai Sambol*

Version 1.0, June 10, 2025

1 Introduction - What is CharLES?

CharLES is an unstructured CFD solver, particularly designed to perform wall-modeled large eddy simulation (WMLES) of flows around complex geometries. It uses a low numerical dissipation scheme (KEEP) and highly parallelized code, is quite user friendly and has been validated over various flow regimes. It was developed by Cascade Technologies, a spin-off from the Center for Turbulence Research (CTR) at Stanford. Cascade Technologies was purchased by Cadence Design Systems sometime around 2022.

Creating simulations with CharLES consists of five steps:

1. Defining the geometry of interest in an `.stl` file;
2. Preparing the geometry using `Surfer`;
3. Generating an appropriate mesh using `Stitch`;
4. Running the solution in the `CharLES solver`;
5. Post-processing the results.

In this manual we will cover steps 2-4, under the assumption that step 1 has already been completed. We will work in both the terminal and the CharLES GUI called `CharLES Connect`.

2 Setup

2.1 Installing CharLES

Multiple parts of the setup will likely need Michael Karp's password, or sudo privileges. For that reason, the whole setup should be done together with Michael.

Start by entering your home directory on zeus, then enter the following lines of code to copy CharLES from Michael's directory and install it in your own zeus directory, where you can use it.

*Based on a session by Ori Haber that took place on 05/05/25

```
$ mkdir CharLES
$ cd CharLES
$ scp mkarp@zeus:/home/mkarp/CharLES/install/24.2/
      Base_CHARLES24.20.000_lnx86_1of1.run .
$ chmod 700 Base_CHARLES24.20.000_lnx86_1of1.run
$ bash Base_CHARLES24.20.000_lnx86_1of1.run
```

Accept the license agreement and choose the following installation folder:

```
$ /home/[username]/CharLES
```

Where `[username]` should be replaced by your username. Once installation is complete, add the following lines of code to your `.bashrc` file:

```
export CDS_LIC_FILE=5280@lm.technion.ac.il
export PATH=$PATH:/home/[username]/CharLES/latest/bin
```

Where `[username]` should be replaced by your username. Finally, source the new `.bashrc` to ensure the commands are updated in the current session (this will occur automatically in later sessions):

```
$ source .bashrc
```

Next, you will need to install `CharLES Connect` on your **local** computer. To do that, go to your home directory on the local computer. Then use the following lines of code to create a folder, copy a file from zeus into your local computer, and install the software.

```
$ mkdir CharLES
$ cd CharLES
```

For SUSE (Red Hat) Linux distributions, use:

```
$ scp mkarp@zeus:/home/mkarp/CharLES/install/24.2/
      Base_CHARLES24.20.000_charles_connect-2024.2.x86_64.rpm .
$ sudo rpm -i ./Base_CHARLES24.20.000_charles_connect-2024.2.x86_64.rpm
```

For debian (Ubuntu) Linux distributions, use:

```
$ scp mkarp@zeus:/home/mkarp/CharLES/install/24.2/
      Base_CHARLES24.20.000_charles_connect_2024.2_amd64.deb .
$ sudo dpkg -i Base_CHARLES24.20.000_charles_connect_2024.2_amd64.deb
```

If any dependencies are missing, install them using `yum` in Red Hat or `apt` in Ubuntu:

```
$ sudo yum/apt install [dependency_name]
```

You should now see the `CharLES Connect` logo on your home screen (Figure 1). Open the app, it should look similar to Figure 2.

Press on the four disks at the end of the home line, which is the button for 'resources'. Press the '+' sign to add a resource. Enter `zeus` as the nickname, `zeus.technion.ac.il` as the host, and then your username and password for zeus. Save the resource and press the refresh button next to the resource button.

You should now see a window for 'zeus' open underneath the 'Home' line. It should be green, which means that a connection has been established.

Figure 1: CharLES Connect Logo

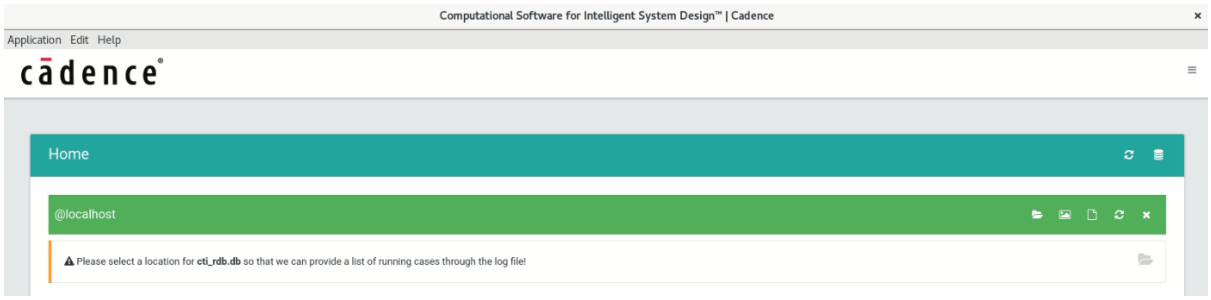

Figure 2: CharLES Connect Home Screen

2.2 Copying Documents for Test Case

In this manual we will practice using CharLES on a test case of flow around an axisymmetric geometry, with a hemispherical head, cylindrical body, and flare.

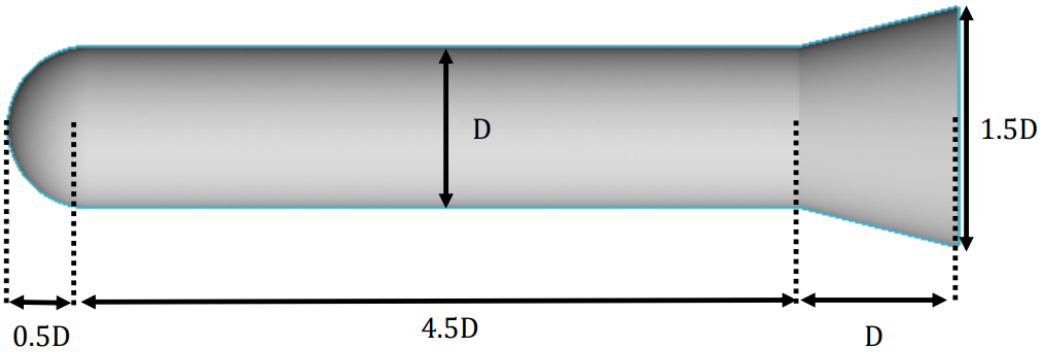

Figure 3: Test Case

In order to do that, we will copy some prepared files from Ori Haber's directory into one of our directories. From now on we will be staying on the **zeus** server, and only tracking our progress from the local computer. Make sure that you are in the `CharLES` directory in your zeus account, and run the following command:

```
$ cp -r /home/ori.haber/CharLES/FPLworkshop .
```

Great! Now you are all set up, and we can begin our tutorial.

3 Running a Test Case

let us start by getting familiar with the test case files that we just copied into our directory, and preparing our stl files. All the files are in a directory called `FPLworkshop` so start by entering that directory.

You should now see three directories. The first two - `M2_Re1K` and `M5_Re100K` - are meant for step 4 when we run the `CharLES solver`, and contain instructions for different run conditions (i.e. different Mach and Reynolds numbers). The third directory - `meshing` is the one in which we will do steps 2 and 3 - running `Surfer` and `Stitch`. Enter that directory now.

3.1 Step 1 - Stl Files

This directory contains the files for running `Stitch`, and we will see them soon. In the meantime, move two more directories inwards:

```
$ cd ./geometry_prep/StlFiles
```

Here you should see two things: A python code called `process_stl.py` and a zip file `StlFiles.zip`. Start by unzipping the latter:

```
$ unzip StlFiles.zip
```

This should result in four new files in your directory: `base.stl`, `cylinder.stl`, `flare.stl`, and `nose.stl`. These are the geometry files that we are going to use in our simulation.

If you now open one of the stl files, or run `head base.stl` to see just the start of the file, you will see that the file start right away with the surfaces that make up the geometry. Once we run `Surfer` each surface will be assigned a name, and that name will be determined by how the file containing the surface starts. For that reason, we would like each file to start with an appropriate name. In order to do that, run the python code:

```
$ ./process_stl.py .
```

This code adds a line to each stl file giving it a header. Open again the same stl file, or run `head base.stl`, and you will see that this time it starts with the words `solid base`. Meaning the file should be treated as a solid geometry, with the name `base`.

Now our stl files are ready for use. Move out one directory, so that you will be in the directory `geometry_prep`.

3.2 Step 2 - Preparing Geometry Using Surfer

We are about to run the `Surfer` step to prepare our geometry by combining the four stl files into one file, and adding the borders of the computational domain. Before we run the code `run-surfer.sh`, let us look at the input that it takes. The input is in the file `surfer.in`, so open that file using whatever reader you prefer (like nano/vi/vim). We will go over the lines that appear in this file, although not in the order they appear in.

The first lines define for the program which stl files it is supposed to use, and that they should each be treated as surfaces (`SURF`):

```
SURF STL StlFiles/base.stl
SURF STL StlFiles/nose.stl
SURF STL StlFiles/cylinder.stl
SURF STL StlFiles/flare.stl
```

For the moment, comment out the line `FLIP ALL` by adding a hash (#) at the beginning of the line. We will understand soon what this line does.

The next important lines are:

```
SURF BOX -1 40.00 -30.00 30.00 -30.00 30.00
MOVE_TO_ZONE NAME FF SUBZONES 4,5,6,7,8,9
```

The first of these defines a box of surfaces between the points $(x, y, z) = (-1, -30, -30)$ and $(x, y, z) = (40, 30, 30)$, where the syntax is

```
SURF [surface_type] [x_min] [x_max] [y_min] [y_max] [z_min] [z_max]
```

The second line takes these surfaces, which are numbered 4-9 (the surfaces taken from the stl files got the numbers 0-3), and assigns them the name `FF` which stands for 'Far Field'. That is what these surfaces are meant as – the far field that encloses the space around our geometry, which is the limit of the area that we want to solve in. It is important to define these surfaces in this input file, and giving them one name allows us to give all of them the same boundary condition.

The last line of our file is:

```
WRITE_SBIN ../fpl_charles_geometry.sbin
```

which tells `Surfer` what the output filename should be (and where it should be located).

Aside from those lines, which are the core parts of how `Surfer` will run, one of the things that we would like is to be able to see the results of the run, before we move on to the next stage. There are two ways to do this. The first is by adding the line `INTERACTIVE` at an appropriate place in the code (after the surfaces we want to view have been defined). This will create an interactive model that we can move around, even though you will see that it is sometimes slow and limited. Uncomment the first of these `INTERACTIVE` lines so that after we run `Surfer` you will be able to see it. The second way is by adding the lines:

```
WRITE_IMAGE NAME=cutPlane_y     GEOM PLANE 3.0 0.0 0.0  0.0 -1.0  0.0
    UP = 0 0 1 WIDTH=20 SIZE=1920 1200 NO_GEOM_BLANKING
WRITE_IMAGE NAME=cutPlane_y_neg GEOM PLANE 3.0 0.0 0.0  0.0  1.0  0.0
    UP = 0 0 1 WIDTH=20 SIZE=1920 1200 NO_GEOM_BLANKING
WRITE_IMAGE NAME=cutPlane_z     GEOM PLANE 3.0 0.0 0.0  0.0  0.0  1.0
    UP = 0 1 0 WIDTH=20 SIZE=1920 1200 NO_GEOM_BLANKING
WRITE_IMAGE NAME=cutPlane_z_neg GEOM PLANE 3.0 0.0 0.0  0.0  0.0 -1.0
    UP = 0 1 0 WIDTH=20 SIZE=1920 1200 NO_GEOM_BLANKING
```

```
WRITE_IMAGE NAME=cutPlane_x     GEOM PLANE 0.0 0.0 0.0  1.0  0.0  0.0
    UP = 0 0 1 WIDTH=10 SIZE=1920 1200 NO_GEOM_BLANKING
WRITE_IMAGE NAME=cutPlane_x_neg GEOM PLANE 0.0 0.0 0.0 -1.0  0.0  0.0
    UP = 0 0 1 WIDTH=10 SIZE=1920 1200 NO_GEOM_BLANKING
```

These lines will create still images with different cuts of the geometry we have created. Each image gives only a partial view, but a thoughtful choice of cuts will give a clear view of the whole geometry in a quick and accessible way.

Before we run `Surfer` there is one more thing we need to do. Close the file and open instead the file `run-surfer.sh`. In the following row, switch `ori.haber` with your own username.

```
#PBS -M ori.haber@campus.technion.ac.il
```

We are now ready to run `Surfer`. Close the file, and run:

```
$ qsub run-surfer.sh
```

This may take a few seconds. Run

```
$ qstat -awn1u [username]
```

to make sure that your program status is `R` for running and not `Q` for 'in the queue' (replace `[username]` with your own username).

Now go back to the `CharLES Connect` GUI that should be open on your local computer. You should see the new `Surfer` job appearing there, with four buttons on the right, as in Figure 4.

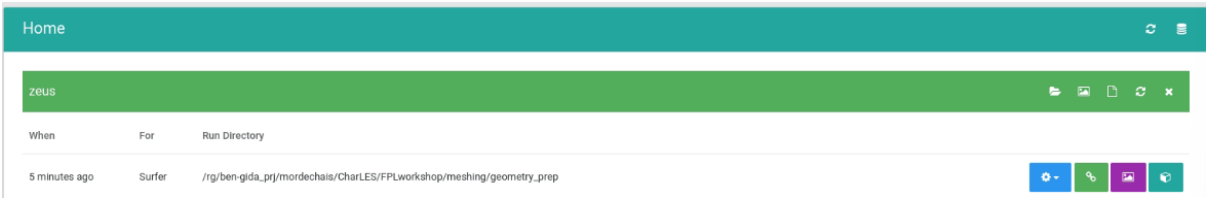

Figure 4: CharLES Connect with a `Surfer` Job

The third button 'image sets' should let you view the images that `Surfer` captured. The fourth button '3d app' should let you view the geometry in an interactive way. Try opening the interactive viewer now, note it may take some time to load.

Some things worth knowing about the interactive viewer: It is not like a regular CAD viewer, in the sense that it is not actually showing the whole geometry. Instead, it generates an image based on the view you set up. If you change that view, it will take the viewer a few seconds to update, and until then it will look like there is a blank space where there should be geometry.

Also, it is important to note the **color** of the geometry. In our case, your geometry should be **gold**. All surfaces in `Surfer` are defined by their normal. The normal of a gold surface

is pointing **inwards**. This is not actually what we want though - we need the normal of our surfaces to point outwards. So we will stop the run of `Surfer` by creating a file called `killsurfer`:

```
$ touch killsurfer
```

Run `qstat` (as above) to make sure that `Surfer` has stopped. The file `killsurfer` should automatically be deleted so that you can run `Surfer` again (this may take a minute or two). Before you do that, open the file `surfer.in` and uncomment the line `FLIP ALL`. This will flip all our surfaces so that their normals point outwards.

Now you can run `run-surfer.sh` again, open the interactive viewer and make sure that all the surfaces have changed their color to silver. Or at least the surfaces `base`, `nose`, `cylinder` and `flare`. The far field surfaces should still be gold, so that their normals point into the domain being simulated.

Now that we have understood the different ways to view the results, let us do our 'actual run'. Stop the present run by creating the file `killsurfer`, open the file `surfer.in` and comment out the line `INTERACTIVE`, then run `Surfer` again. This time it should run and terminate pretty quickly. No interactive viewer can be used, but now our results have been saved to the designated file `../fpl_charles_geometry.sbin`.

Congratulations - you have finished Step 2 - Preparing the geometry using `Surfer`.

3.3 Step 3 - Generating a Mesh Using Stitch

`Stitch` is the CharLES mesh generation tool. It uses hexahedral elements ordered using a Voronoi diagram, which provides zero angle between cell faces and cell centers.

In order to run this step, move out one directory, so that you are in the directory `meshing`. There should be three files in this directory: The output file from `Surfer`, the file `stitch.in`, and the file `run-stitch.sh`. Just like we did with `Surfer`, let us look at the input file before running `Stitch`. This input file has many more built in comments, but we will still go over the different lines and what they do.

The first line defines for the program which file contains the output of `Surfer` (Step 2):

```
PART main SURF SBIN fpl_charles_geometry.sbin
```

The resolution of the mesh to be created is determined by two parameters: `HCP_DELTA` which defines the characteristic length of the largest cell (in the far field), and γ which is an integer that is defined for every area in which we want a refined mesh. γ defines the refinement of the mesh in that area relative to the far field, using the relation:

$$\Delta\text{Refinement} = \texttt{HCP\_DELTA} \times 2^{-\gamma}$$

The parameter γ is unit-less, and the parameter `HCP_DELTA` uses the units defined in the original stl files.

That is why the next lines of our code define these parameters. First, we define `HCP_DELTA`:

```
HCP_DELTA 50
```

Then we create three variables for the different values of γ that we will want to use:

```
DEFINE WALLS 10
DEFINE SHOCK 8
DEFINE FLARE-WAKE 8
```

After that, we define two areas in which we want extra refinement - the space around the leading edge where we expect a bow shock, and the space around the trailing edge where we expect a wake. In both spaces we have chosen to create a truncated cone shape of refined mesh:

```
HCP_WINDOW TCONE -0.75 0.0 0.0 0.25 1.25 0.0 0.0 2.0
        LEVEL $(SHOCK)       N=5 NLAYERS 2
HCP_WINDOW TCONE  3.5  0.0 0.0 0.5  8.0  0.0 0.0 1.5
        LEVEL $(FLARE-WAKE) N=5 NLAYERS 2
```

The syntax here is:

```
HCP_WINDOW [shape] [base_center_x] [base_center_y] [base_center_z]
           [base_radius] [top_center_x] [top_center_y] [top_center_z]
           [top_radius] LEVEL [level] N [n] NLAYERS [nlayers]
```

Notice that in both we have used the γ variables we chose previously, in order to determine how refined we want the mesh in those areas. The parameters `LEVEL`, `N`, and `NLAYERS` are explained in the next paragraph.

We also define the mesh refinement on each of the borders - each of the inner surfaces:

```
HCP_WINDOW FAZONE base           LEVEL $(WALLS)    N=10 NLAYERS 10
HCP_WINDOW FAZONE nose           LEVEL $(WALLS)    N=10 NLAYERS 5
HCP_WINDOW FAZONE cylinder       LEVEL $(WALLS)    N=10 NLAYERS 5
HCP_WINDOW FAZONE flare          LEVEL $(WALLS)    N=10 NLAYERS 5
```

The parameter `LEVEL` is the value of γ for the refinement of the mesh **on** the chosen surface. The parameter `N` is the thickness of the mesh on the chosen surface, i.e. the amount of cells extending from that surface that have the same level of refinement as the mesh on the surface. We will call this one layer of the mesh. The parameter `NLAYERS` determines the thickness of every consecutive layer (meaning every following layer, from the end of the first layer outwards). Notice that from one layer to the next the mesh becomes half as refined, until it reaches the same refinement as the outermost layer, which is `HCP_DELTA`. We also add the command `NSMOOTH 25` which smooths out the passage from one layer to the next.

Finally, we save images of the created mesh using

```
WRITE_IMAGE NAME=cutPlane_nose_w2  GEOM PLANE 0.6 0.0 0.0 0.0 -1.0 0.0
    UP = 0 0 1 VAR=mesh WIDTH=2.0  SIZE=1920 1200
    NO_GEOM_BLANKING HIDE_ZONES_NAMED FF
WRITE_IMAGE NAME=cutPlane_base_w4  GEOM PLANE 6.0 0.0 0.5 0.0 -1.0 0.0
    UP = 0 0 1 VAR=mesh WIDTH=4.0  SIZE=1920 1200
```

```
    NO_GEOM_BLANKING HIDE_ZONES_NAMED FF
WRITE_IMAGE NAME=cutPlane_full_w5  GEOM PLANE 1.8 0.0 0.0 0.0 -1.0 0.0
    UP = 0 0 1 VAR=mesh WIDTH=5.0  SIZE=1920 1200
    NO_GEOM_BLANKING HIDE_ZONES_NAMED FF
WRITE_IMAGE NAME=cutPlane_full_w10 GEOM PLANE 1.8 0.0 0.0 0.0 -1.0 0.0
    UP = 0 0 1 VAR=mesh WIDTH=10.0 SIZE=1920 1200
    NO_GEOM_BLANKING HIDE_ZONES_NAMED FF
WRITE_IMAGE NAME=cutPlane_full_w20 GEOM PLANE 1.8 0.0 0.0 0.0 -1.0 0.0
    UP = 0 0 1 VAR=mesh WIDTH=20.0 SIZE=1920 1200
    NO_GEOM_BLANKING HIDE_ZONES_NAMED FF
```

and write the results of the mesh to a restart file using

```
WRITE_RESTART grid.mles
```

It is important to note that saving the mesh to a file is a time-consuming process, so first you should run the program and modify parameters until you are happy with how the resulting mesh looks in the images, and only after that write the mesh to a file. You can also use the interactive viewer if you wish.

Before running `Stitch`, remember to update your username in the `run-stitch.sh` file. Then you can run `Stitch`.

```
$ qsub run-stitch.sh
```

You should now see a `Stitch` job in `CharLES Connect`. You may need to reload and to scroll to see it.

Give the program a few seconds to run, then take a look at the resulting mesh. It will likely be easier for you to look at the mesh through a different viewer and not `CharLES Connect` (for example, using an sftp connection and `get` to copy the images to your local computer), but it can be done through `CharLES Connect` as well. You should get a result that looks like Figure 5.

You will notice that we see in this picture a refined mesh close to the body and a less refined mesh far from the body, but we don't see the two truncated cone areas of refined mesh that we had wanted to add. That is because they are 'swallowed' inside the other refined mesh areas. Try opening `stitch.in` and changing `HCP_DELTA` to 10, and then run `run-stitch.sh` again. This time your resulting mesh should look like Figure 6.

Why is this so? Changing the value of `HCP_DELTA` makes all the cells smaller. This means that A) more mesh stages are needed in the transition from the boundary layer to the far field, and B) each layer 'shrinks' and covers less of the space. This means that the truncated cone areas of extra refinement are revealed, since they stay in the same space, but now the mesh around them doesn't have the same level of refinement as they do, and therefore they stand out.

If you are interested in seeing the size of your mesh (important for understanding the expected size of the `CharLES solver` results), run the command:

```
$ grep ncv stitch.log
```

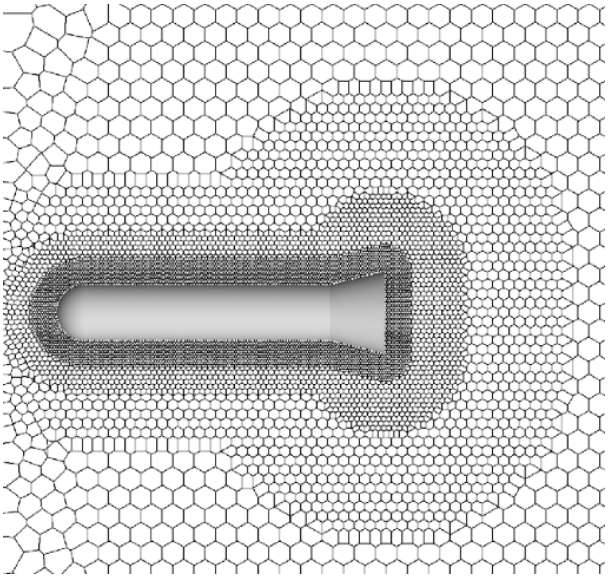

Figure 5: First Mesh

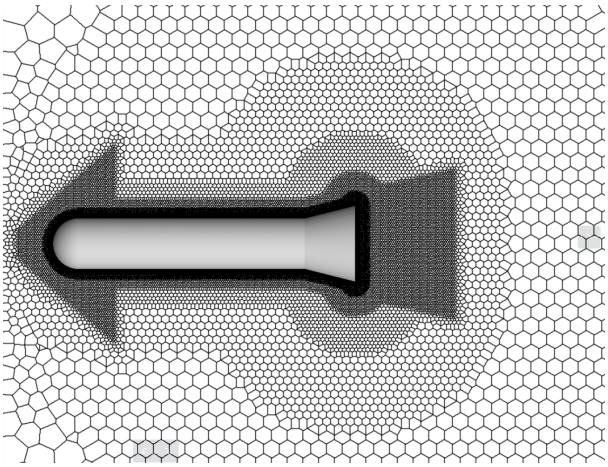

Figure 6: Second Mesh

Where `ncv` stands for the number of control volumes.

That's it - we have finished Step 2 - Generating a Mesh Using `Stitch`.

3.4 Step 3 - Running the CharLES solver

For the next step, go out one directory and then go into the directory `M2_Re1K`. Notice that when we finished the `Surfer` step the output was already in the same directory as the code for the `Stitch` step. But the output from the `Stitch` step was put into the same directory as the `Stitch` step, and not into the directory we are presently in! This is so that we can use the same mesh for simulations with different run conditions. The present directory is meant for the run condition of $M = 2;\ Re = 1000$, whereas a different directory was made for conditions of $M = 5;\ Re = 100,000$. But in both we can use the same mesh.

In order to do that, let us add to the present directory a symbolic link to the mesh file:

```
$ ln -s ../meshing/grid.mles
```

Now that we have the necessary mesh, let us look at the other input file - `CharLES_ig.in`. The addition `ig` to the name of this file is because it is set up to run the `CharLES solver` assuming that air acts like an **ideal gas**. The name of the file itself is not important, the actual choice to use ideal gas equations is done inside the file (will be explained soon). We will describe this file in a much more general way, without going into each and every line.

The first lines of the file deal with creating variables for the parameters of the problem - the center of gravity of the geometry, the fluid properties, the reference state (fluid properties at ∞), and the typical time of the problem. This part ends with defining the angle of attack, and according to it the free flow far from the geometry. Since we are using an angle of attack 0 in this example, the free flow is simply the speed at ∞ in the x direction.

After defining all those variables, the next part chooses the relevant variables as the parameters of the problem. It also chooses the EOS (which Equations Of State should be used), and this is where we choose `IDEAL_GAS`. But there are more options that can be used.

Next we specify whether we want a sub-gridscale model, meaning if we want the solver to model the turbulent effects that are smaller than the size of the grid. If you are interested in running CharLES as an LES solver, a model should be chosen here (as that is how LES solvers work). A model that can be used is `VREMAN`. In this example we will run CharLES as a **DNS** solver, and therefore we can set this parameter to `NONE`.

In the line `STAT` we choose which parameters interest us, i.e. which parameter's values should be written to the output file, or plotted when requested.

Following that we choose the boundary conditions we wish to use. In this example, we have chosen to make all the walls `ADIABATIC`, and the far field such that it matches the conditions at ∞. In the case of using the `CharLES solver` for LES, use `WM_ALG_ADIABATIC` (Wall Model Algebraic Adiabatic), which is a model for solving boundary layers with large cells.

Then we write down the interval size in which the requested parameters should be measured and the initial conditions.

After that we define two time related parameters for our run, and there are a few different ways to do this:

The first time-parameter is the end condition - at what time should the solver stop. This can be defined in **real** time using `RUNTIME` or in **simulation** time using `SIMTIME`. So, for example, setting `RUNTIME 1` means that the program will run for exactly one hour. Note that **real** time is measured in hours, whereas **simulation** time is measured in non-dimensional time units for a non-dimensional model, or **seconds** for a dimensional model.

Alternatively, we can tell the program how many steps we want it to make using `NSTEPS`.

The second time-parameter is the time step. This can be defined as a constant step size using `TIMESTEP DT` or as a changing step size using `TIMESTEP CFL`.

Finally we write the results that interest us: Forces upon the different surfaces, moments around specific points (these are included at the end of the command `WRITE_FORCE` and are saved into separate files), temperature and pressure gradients measured at certain points in the domain (according to a csv file in the present directory), images of the flow that we wish to be captured, and the calculated residuals. Each of these is saved into a different folder or file.

That's it for the `CharLES_ig.in`, now let us run the solver. This will take some time, but you can shorten the time by choosing different time variables as discussed. Remember to update your username in the `run-charles.sh` file.

```
$ qsub run-charles.sh
```

4 Part 4 - Post-Processing the Results

Congratulations! You have successfully run your first CharLES simulation. Take a look in the `CharLES Connect` GUI to see videos of the flow you simulated. Your results should look similar to those appearing in Figure 7. Each of these images shows a different variables - like u, v, w, p, ρ, T.

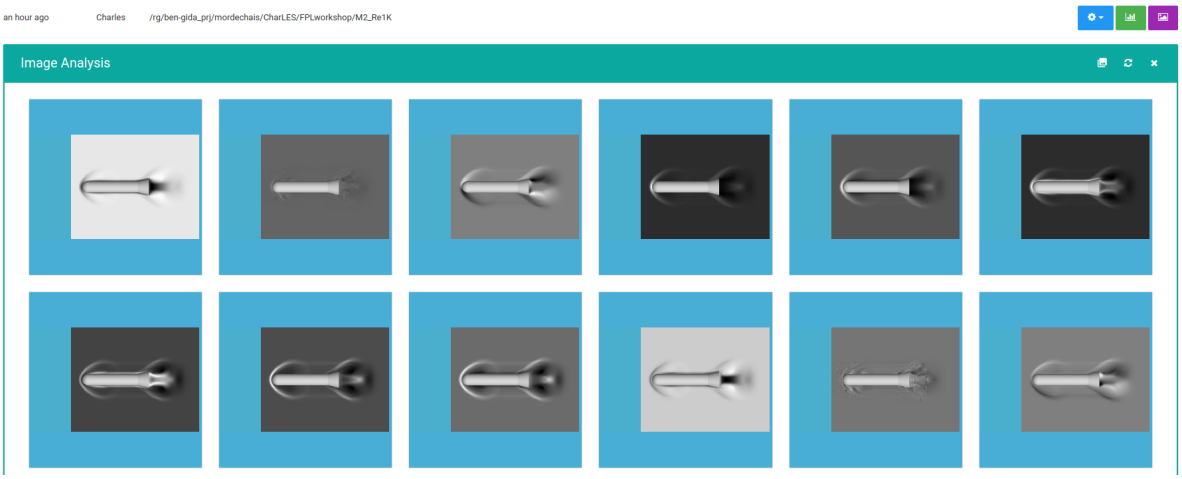

Figure 7: Resulting Flowfields

To see a video, hover over one of the images. Press on the button that appears on the top right of the image, then 'LOAD 152 IMAGES', and watch as the flowfield develops.

All that is left to do now is to analyze the results. If you ask Ori, he will send you a simple code for reading probed data into MATLAB, for use in a simple baseflow analysis.

5 Some Additional Comments

- CharLES is built for **compressible** simulations. If you are interested in using it for an incompressible simulation, the way to do it is to choose a small Mach number. Note that for very small Mach numbers the equations may become stiff, which will require very small time steps, usually we will not go lower than $M = 0.1$.

- Running `Surfer` is quite simple, so it can be done on a single core. `Stitch` can be run on a small amount of cores, unless an especially large mesh is being generated, in which case it is recommended to use a larger amount of cores. `CharLES Solver` is the heavy duty program that needs to be run on many cores.

- How big can we expect our results to be? That depends on how much info we wish to save. If you are saving images, forces and moments at select points, it will take little space. If you are interested in a baseflow it will take significantly more space, but you only do that once. In general it can be said that as long as you are not measuring too many things the solution will take up slightly more space than the size of the mesh. The size of the mesh can be checked by opening `/meshing/stitch.log`.

6 Cleaning up

Now that you have finished the tutorial, we recommend deleting all the results in order to free up space for future runs. Make sure you are in the `M2_Re1K` directory and run:

```
$ bash clean_dir.sh
```

This is a short code that will delete all the results of the run, while leaving all the input and run files so that they can be reused.